

LUXORA · Ship It

Push to GitHub, spin up a free database, deploy, seed, verify — the 20-minute path from a finished local repo to a public URL. Every command below is copy-pasteable.

repo: luxora · 22 commits · MIT + CC photo attributions · docs/DEPLOYMENT.md holds the prose version

The whole trip, at a glance

Nothing here is optional except Route B/C alternatives. Times are real-world medians for this stack.

1	Push to GitHub	rename branch main · create empty repo · push — CI starts itself	~3 min
2	MongoDB Atlas	free M0 cluster · DB user · allow 0.0.0.0/0 · copy SRV URI	~5 min
3	Render deploy	New → Blueprint → paste MONGO_URI as secret → create	~6 min
4	Seed once	npm run seed against the Atlas URI — hotels, bookings, reviews	~1 min
5	Verify + harden	health, sign-in, full booking, admin · change demo passwords!	~5 min

Already done for you — the repo is push-ready today

22 logical commits · .gitignore keeps .env / node_modules / dist out (only .env.example is tracked) · README with full API reference · MIT LICENSE + ATTRIBUTIONS.md for the CC photos · .github/workflows/ci.yml (every push: lint, build, seed into a real MongoDB service container, 69 API smoke checks, QR round-trips, Playwright over 19 routes incl. auth sessions) · render.yaml blueprint so the deploy pre-configures itself · multi-stage Dockerfile for any other host.

1 · Push to GitHub

Option A — GitHub CLI (one auth flow, no tokens to paste)

```
gh auth login          # browser sign-in; pick HTTPS + gh as credential helper
cd luxora
git branch -M main
gh repo create luxora --public --source=. --push
```

Option B — website + git commands

```
# github.com -> New repository -> name 'luxora' -> LEAVE all three checkboxes UNCHECKED
cd luxora && git branch -M main
git remote add origin git@github.com:USERNAME/luxora.git # or https://github.com/...
git push -u origin main
```

- No SSH key yet? Settings → SSH and GPG keys → New SSH key, paste `cat ~/.ssh/id_ed25519.pub` (create with `ssh-keygen -t ed25519`). With HTTPS instead, GitHub wants a Personal Access Token (scope: repo) as the password — or just use Option A.
- Pre-flight (should print NOTHING): `git ls-files | grep -E '\.env$|node_modules|client/dist'` — the repo already passes; re-run if you change .gitignore.
- Final local confidence: `npm run verify` (lint + build + 69 API + 19 UI routes).
- Polish once pushed: repo About = “Premium hotel booking — React + Vite + Tailwind + Framer Motion · Express + MongoDB + JWT · full-stack” · topics: `react mongodb express tailwind framer-motion fullstack nodejs booking` · add 2–3 screenshots under docs/ and link them in the README.

The moment you push

GitHub Actions wakes up and reruns your entire verification suite on a clean Ubuntu + MongoDB 7 — lint, build, seed, 69 API checks, QR decode round-trips, and a headless-Chromium tour of 19 routes signed in as both guest and admin. Every future commit you deploy is checked by the same battery. Add the badge to README: `[![CI](https://github.com/YOU/luxora/actions/workflows/ci.yml/badge.svg)](https://github.com/YOU/luxora/actions/workflows/ci.yml)`

2 · Database — MongoDB Atlas (free M0, no card)

1. `mongodb.com/atlas` -> sign up -> Create new project (name: luxora)
2. Build a Database -> FREE tier (Shared M0, 512 MB) -> pick region near your host
3. Security -> Database Access -> Add New Database User
username: luxora_app password: use the generated one -> COPY IT NOW
4. Security -> Network Access -> Add IP Address -> Access List Entry: 0.0.0.0/0
5. Database -> Connect -> Connect your application -> Drivers (Node.js) -> COPY the string:
`mongodb+srv://luxora_app:PASSWORD@cluster0.XXXXX.mongodb.net/luxora?retryWrites=true&w=majority`

- That string is MONGO_URI — the only connection config the app needs. Database name luxora in the path is what we use.
- Why 0.0.0.0/0 for a demo: Render/Railway egress IPs rotate, so CIDR pinning breaks within days. The DB user has a strong generated password and the public never talks to it directly — only your JWT-authenticated API does. If the URI ever leaks: Atlas → edit user → rotate password.
- Special chars in the password? Percent-encode them inside the URI (or don't use special chars). Atlas validates `?retryWrites=true&w=majority` is present.
- Paid plan later: resize M0 → M2 in one click; URI does not change (same cluster).

3 · Hosting — pick a lane (A = the zero-config one)

Route A · Render free tier — this repo ships the blueprint

1. `render.com` -> sign up WITH GITHUB (authorises the repo)
2. Dashboard -> 'New +' -> Blueprint -> select repo 'luxora' -> it reads `render.yaml`
3. Only the blank/'sync:false' fields need you:
MONGO_URI = the Atlas string -> click 'Secret'
SEED_ADMIN_PASSWORD = something new -> Secret
SEED_DEMO_PASSWORD = something new -> Secret
CORS_ORIGIN, CLIENT_URL = `http(s)://luxora-xyz.onrender.com`
(Render URL exists only after create: leave both, then set them in Environment tab + restart)
4. Apply -> auto build 'npm ci && npm run build' -> start 'npm start' -> healthcheck `/api/v1/health`
5. Visit `https://luxora-xyz.onrender.com` -> the site.

- Why it's this simple: one process serves everything — Express exposes `/api/v1` and the built SPA (`SERVE_CLIENT=true`). Same-origin in production, so cookies/refresh tokens just work and there's no second service to wire.
- Blueprint also: generates distinct 32+ char JWT secrets (prod boot hard-fails on weak ones — deliberate), sets Node 20 via engines, auto-deploys on every push to main.
- Free tier sleeps after ~15 min of quiet → first visit takes 30–60 s. Fine for a portfolio; \$7/mo Starter removes it. Or keep-alive: an UptimeRobot/cron GET to `/api/v1/health` every 10 minutes.
- Manual form (if Blueprint flow changed upstream): New Web Service → repo → Runtime Node · Build `npm ci && npm run build` · Start `npm start` · Health check path `/api/v1/health` → same env vars.
- Custom domain later: Render → Settings → Custom Domain → point a CNAME at the assigned target → TLS issues automatically → update `CORS_ORIGIN` + `CLIENT_URL`.

Route B · Railway — same app, no cold starts, trial credit instead of free tier

```
npm i -g @railway/cli && railway login
cd luxora && railway init && railway up          # builds via package.json scripts
railway variables --set MONGO_URI='mongodb+srv://...' \
  --set JWT_SECRET=$(openssl rand -hex 32) --set JWT_REFRESH_SECRET=$(openssl rand -hex 32) \
  --set NODE_ENV=production --set SERVE_CLIENT=true \
  --set CORS_ORIGIN=https://URL.onrender-or-railway.app --set CLIENT_URL=https://SAME
railway open                                     # generate a public domain first, then set those tw
o
```

Route C · Any VPS (Hetzner/Fly/Lightsail, ~\$5/mo) with the shipped Dockerfile

```
# local: build + push image to ghcr/docker hub, or build on the box
docker build -t luxora .
docker run -d --name luxora --restart unless-stopped \
  -p 127.0.0.1:4000:4000 --env-file .env.production luxora

# .env.production (same keys as .env.example):
# MONGO_URI, JWT_SECRET, JWT_REFRESH_SECRET (>=32 chars, distinct),
# NODE_ENV=production, SERVE_CLIENT=true, CORS_ORIGIN, CLIENT_URL

# TLS in two lines – Caddy (apt install caddy):
yourdomain.com {
  reverse_proxy 127.0.0.1:4000
}
```

- The image is stateless except the uploads folder (mount -v luxora-uploads:/app/uploads) — or configure Cloudinary and admin uploads go to the CDN instead.
- No-Docker variant works too: `npm ci && npm run build` once, then a systemd unit running `node server/src/index.js` with the env vars above. One process. That's the deployment.
- Admin image uploads return a clean 501 until `CLOUDINARY_*` keys exist; imagery for the demo world ships inside the repo, so nothing is blocked.

4 · Seed the live database — once

```
# from your laptop (the Atlas URI must allow your IP – 0.0.0.0/0 covers it):
MONGO_URI='mongodb+srv://luxora_app:PASSWORD@cluster0.XXXXX.mongodb.net/luxora?retryWrites=true&w=majority' npm run seed

# or Render dashboard -> your service -> Shell tab: npm run seed
# or Railway: railway run npm run seed

# wipe & rebuild anytime (demo data only!): npm run seed:reset
```

Writes the deterministic demo world the UI expects for review: 16 named hotels across Jaipur, Dubai, Bali, Tokyo, Paris, Santorini and the Maldives · 38 rooms with real price ladders · 45 travellers · ~150 bookings spread with seasonality · 37 written reviews · wishlists · the two demo logins. Skip it for a real launch and create hotels in the admin console instead.

5 · Verify live, then harden before sharing

Check	Do	Expect
API + DB	<code>https://URL/api/v1/health</code>	<code>ok:true</code> · <code>database:connected</code>
SPA routing	open URL/discover, hit reload	page renders — no 404
Guest flow	sign in → search → book 4 steps → confirm	QR pass · folio · download works
Auth session	close tab, reopen	still signed in (refresh cookie rotates)
Admin	log in <code>admin@luxora.travel</code>	stats, 4 charts, CRUD, moderation
CI	GitHub repo → Actions tab	all green on latest push

One warning, non-negotiable

Login page ships demo quick-fill chips for `aarav@me.com / LuxoraGuest#26` and `admin@luxora.travel / LuxoraAdmin#26`. Perfect for reviewers — reckless for a public URL. Before sharing: set new `SEED_ADMIN_PASSWORD / SEED_DEMO_PASSWORD` (Render → Environment → save → restart; seed runs once so also re-run seed OR just change the admin password from the Profile page after login) — or remove the chips in `client/src/pages/LoginPage.jsx` (“Demo accounts” block).

Troubleshooting — known failure modes, pre-diagnosed

Symptom	Cause → fix
Deploy builds fine but every request is 500	mongod unreachable: Atlas Network Access lacks Render's IPs → add <code>0.0.0.0/0</code> , save
Boot log: <code>MongoServerSelectionError</code>	URI typo / missing <code>w=majority</code> / unencoded special chars in password → regenerate password without symbols
“Configuration error: <code>JWT_SECRET</code> must be at least 32 characters”	prod hard-fails by design → paste 64 hex (Blueprint's <code>generateValue</code> already does)
Login OK, refresh logs out	<code>CORS_ORIGIN/CLIENT_URL</code> ≠ actual origin (must include <code>https://</code> , no trailing slash) → fix, restart
Site loads but 0 stays everywhere	you skipped step 4 — seed
First visitor of the day stares at a spinner 60 s	free-tier cold start → upgrade, or keep-alive ping <code>/api/v1/health</code>
Bookings work on laptop but 429 in demos	shared demo IP hits rate limit — <code>RATE_LIMIT_MAX</code> env (default <code>300/min/IP</code>) → raise for load, not for prod

Day-2 loop

- Ship an update: `commit` → `git push` → CI checks → Render auto-deploys the same commit. Rollback = Render dashboard → one click to prior build.
- Backups: `weekly mongodump --uri="$MONGO_URI" --gzip --out backups/` from cron; paid Atlas tiers add native snapshots.
- Watch: Render → service → Logs shows JSON request logs + the one boot warning (“`CLOUDINARY unset...`”) until you add media keys. 500-rate spikes are obvious there.
- Custom domain + HTTPS: ~10 minutes (see step 3 bullet) — remember `CORS_ORIGIN/CLIENT_URL` follow the move.
- Scale past free tier: the app is one stateless node process — a single \$5 box or 1x Render Starter handles demo traffic with headroom; Atlas M0's limit (connections/storage) is the first wall, resize there too.

Prose version with everything explained: `docs/DEPLOYMENT.md` (in the repo). Env reference: `.env.example`. Endpoints + response envelopes: `README.md`. Design decisions: `docs/ARCHITECTURE.md`. Live suites: `npm run verify = lint · build · test:api (69) · test:ui (19 routes) · test:qr`.